

Learning map

See the whole workflow before studying the pieces.



The key principle

Computers receive strings and symbols, but useful NLP depends on linguistic structure and context. Session 1 moves from raw text to tokens, grammatical labels, entities, patterns, counts, and interpretable observations.

Listen for these distinctions

- ☐ human meaning vs machine representation
- ☐ word, character, subword, and sentence tokens
- ☐ syntax, semantics, discourse, and pragmatics
- ☐ POS tags, named entities, and co-reference
- ☐ raw frequencies vs cleaned frequencies

LECTURER EXAMPLES

Core vocabulary

Write the lecturer's example beside each definition.

Corpus

The collection of documents selected for analysis.

Token

A unit processed by a system; it is not always a whole word.

Syntax & POS

Syntax describes grammatical structure; POS tagging labels each token's word class.

Named entity

A person, organisation, location, time, quantity, or other categorised mention.

Semantics

The meaning of words or texts; context helps distinguish senses such as two meanings of "bark".

Discourse

Meaning created across connected sentences rather than within one sentence alone.

Tokenisation is a design decision

The correct token boundaries depend on the task.

RAW TEXT

I can't believe it's not butter. Visit <https://example.com!>

Should “can't” become one token, “can” + “not”, or “ca” + “n't”?

Should punctuation be kept as separate tokens for this task?

How should a tokenizer preserve the URL as one meaningful unit?

Compare a custom regular-expression tokenizer with NLTK `word_tokenize`.

Code reading

Explain each transformation before running it.

```
from nltk.tokenize import sent_tokenize, word_tokenize

sentences = sent_tokenize(mytext)

tokens = word_tokenize(mytext)

tags = nltk.pos_tag(tokens)
```

How are sentence tokens different from word tokens?

Why does NLTK return punctuation as tokens?

How can POS tags support NER, lemmatisation, parsing, or word-sense disambiguation?

Session 1 notebook workflow

Follow the actual order used in the practical notebooks.

1. Collect text

CAPTURES	Python variable, web page, or CSV
LIMITATION	Inspect source structure and access assumptions

2. Analyse language

CAPTURES	Sentence/word tokens, POS, NER, dependency tree, regex
LIMITATION	Outputs depend on tokenisation and pretrained rules

3. Explore a corpus

CAPTURES	Word counts, frequencies, plots, and word clouds
LIMITATION	Frequent terms may be stopwords or punctuation

EXAMPLES / DIAGRAMS FROM CLASS

Cleaning and interpretation

Compare results before and after each preprocessing decision.

☐ Inspect the dataset with `info()`, `describe()`, `head()`, and `sample()`

☐ Measure text length and ask a question that makes the count meaningful

☐ Tokenise the corpus and inspect the raw frequency distribution

☐ Lowercase and remove task-irrelevant stopwords and punctuation

☐ Recalculate frequencies and explain what changed—and what may have been lost

Evidence and interpretation from the notebook

After-class retrieval

Answer from memory, then check and correct in another colour.

1. Why is machine understanding of language difficult? Give two slide examples.

2. Compare word, character, subword, and sentence tokenisation.

3. Explain the difference between syntax, semantics, discourse, and pragmatics.

4. How do POS tagging, NER, and co-reference add structure to raw text?

5. Why should word frequencies be compared before and after cleaning?

ONE-SENTENCE SUMMARY
